

INFO-F-201 — Systèmes d'exploitation

Introduction au C

Joël Goossens (N8.107) Olivier Markowitch (N8.106)

Arnaud Leponce (N8.213) Attilio Discepoli (N8.210)

Alexis Reynouard (N8.215)

1 Pourquoi le C ?

1.1 Un programme en langage machine

Imaginez que nous voulons construire un processeur simple avec trois espaces mémoires, appelés *gauche*, *droite* et *résultat*. On peut connecter *gauche* et *droite* à un circuit électronique qui fait une addition, et stocker ce résultat dans *résultat*. Pour ajouter plus de possibilités, on connecte aussi un circuit de soustraction à *gauche* et *droite*, puis un *sélecteur* qui choisit, selon une commande reçue, quel résultat utiliser stocker dans *résultat*. Par exemple: 1 demande de stocker le résultat de l'addition et 2 celui de la soustraction. Ces numéros sont appelées opcodes (codes d'opération). En écrivant une suite d'opcodes (convertis en binaire), on crée un programme exécutable par la machine.

1.2 L'assembleur: rendre les programmes lisibles

Pour faciliter la programmation, au lieu d'écrire directement ces nombres binaires, on utilise un langage appelé assembleur. On remplace les opcodes par des noms symboliques, par exemple «ADD» pour addition. Chaque processeur a son propre langage assembleur qui correspond aux instructions qu'il comprend. L'assembleur est simplement une couche lisible sur le code machine. Les premiers «compilateurs» assembleur vers code machine étaient des sortes de machine à écrire. Sur le clavier, les touches correspondaient à des opérations. Plutôt que de déposer de l'encre sur une feuille, la machine faisait des trous dans une carte perforée. Ces trous étaient l'opcode en binaire de l'opération voulue.

1.3 L'évolution des langages de programmation

De nombreux autres langages sont ensuite apparus, souvent spécialisés pour un matériel ou un domaine (mathématiques, physique, etc.). Ces langages étaient trop proches du matériel ou trop abstraits, et ne permettaient pas toujours de manipuler les données et les variables avec différentes sémantiques (valeur, référence, alias, pointeur) facilement. En 1969, le langage B a été conçu pour combiner un bon contrôle machine avec un certain niveau d'abstraction. Puis, en 1972, Dennis Ritchie, un des deux ingénieurs de B, a créé le langage C pour réécrire le système d'exploitation Unix, alliant *lisibilité*, *efficacité* et *portabilité*.

1.4 Les forces du langage C

Le langage C adopte un juste niveau d'abstraction: il permet de contrôler précisément la machine tout en offrant un code clair et efficace, utilisable sur différentes architectures. Il offre plusieurs avantages clés :

Sémantique flexible des variables On peut copier une variable (sémantique de valeur) ou prendre son adresse (sémantique de pointeur), permettant ainsi plusieurs noms pour une même donnée.)

Types explicites Chaque variable a un type qui détermine sa taille en mémoire et comment interpréter ses bits. Cela permet au compilateur de choisir l'instruction machine adaptée pour effectuer les opérations, ce qui n'existait pas dans B.

Création de types composés Possibilité de définir des types complexes par agrégation.

Portabilité efficace La taille des types principaux n'est pas rigide, ce qui permet au même code d'être compilé efficacement sur plusieurs machines.

1.5 Le C et les systèmes d'exploitation

C est donc idéal écrire un système d'exploitation (OS). Mais il l'est aussi pour communiquer avec l'OS.

Un OS rend le matériel accessibles via des fonctions appelées appels système. Ces appels sont décrits précisément en termes de mémoire, indépendamment du langage, mais leur utilisation est aisée en C. En pratique, faire un appel système en C revient souvent à appeler une petite fonction qui s'occupe de communiquer avec le noyau de l'OS.

Le code en C est souvent très proche du langage assembleur, ce qui permet d'écrire des couches logicielles très performantes et proches du matériel, tout en restant plus lisible et maintenable que l'assembleur pur. C'est pourquoi la majorité des noyaux modernes comme Unix, Linux, macOS ou Windows sont écrits principalement en C.

2 Variables, adresses et pointeurs

2.1 Sémantique par défaut des variables

En C, lorsqu'on déclare une variable, elle contient une valeur. Par défaut, la sémantique des variables est une emph{sémantique de copie} : cela signifie que quand on affecte une variable à une autre, ou quand on passe une variable en argument à une fonction, la valeur est copiée.

begin{\lstlisting}[caption=Exemple de copie de variable] int a = 5; int b = a; b = 10; end{\lstlisting}

Ainsi, modifier la variable `b` n'a pas d'effet sur `a` puisque ce sont deux emplacements mémoire différents.

2.2 Qu'est-ce qu'une adresse ?

Une emph{adresse} est un numéro unique qui identifie un emplacement en mémoire. Chaque variable occupe un certain emplacement mémoire, et on peut accéder à cet emplacement via son adresse.

On peut obtenir l'adresse d'une variable en utilisant l'opérateur `&`. On peut aussi parler de ce qui est à une adresse avec l'opérateur `*`.

```
begin{lstlisting}[caption=Obtenir l'adresse d'une variable] int x = 3; printf(« Adresse de x : %pn »,
(void)&x); (&x) = 4; end{lstlisting}
```

2.3 Pointeurs

Un `pointeur` est le type d'une variable qui stocke une `adresse` mémoire. Ou plutôt, une variable qui stocke l'adresse d'un entier doit être de type «pointeur vers un entier». Ceci s'écrit: `int*`. «Pointeur» et «adresse» sont donc synonyme.

Quand l'étoile `*` est utilisée après un `type` `X`, elle signifie: «pointeur vers `X`» / «adresse d'un `X`». C'est la déclaration d'un type. Ce n'est pas une opération. Par exemple, **pointeur vers `int`**: `int*`.

Mais quand l'étoile est utilisée avant une `adresse` `Y`, elle signifie: «ce qui se trouve à l'adresse `Y`». (On l'appelle, opérateur de déréférencement.) C'est ce qu'on a fait dans l'exemple précédent. C'est une opération. Donc, devant un pointeur (qui est une adresse), l'étoile permet d'accéder à la valeur pointée.